

Préparation des 45 minutes de questions

Date : 2026-07-10. Réponses volontairement courtes (2-4 phrases), honnêtes sur les limites, chacune avec une preuve à montrer si le jury creuse. Conseil : reformuler la question avant de répondre, et ne jamais survendre — « c'est une limite assumée, voici le plan » vaut mieux qu'une esquivé.

Choix technologiques

Pourquoi pas de framework front (React, Vue...) ? Le besoin est un site d'annonces où chaque page est indexable et rapide : le rendu serveur Twig répond mieux (SEO naturel, pas d'état client complexe). Le JavaScript n'est ajouté que là où il apporte quelque chose (carte, pad de signature, graphiques, autocomplétion), sous CSP stricte. Un framework front aurait ajouté du poids et une surface d'attaque sans bénéfice utilisateur. *Preuve* : `public/js/` (5 scripts ciblés), `views/`, CSP dans `src/app.js`.

Pourquoi des sessions plutôt que du JWT ? Les sessions en base sont révocables immédiatement (suspension d'école, réinitialisation de mot de passe qui déconnecte partout), tiennent dans un cookie HTTP-only inaccessible au JavaScript, et il n'y a aucune API tierce à servir. Un JWT ne se révoque pas sans réintroduire... un stockage serveur. *Preuve* : modèle `Session`, `test/ameliorations-v2.cjs` (invalidation).

Votre application est-elle orientée objet ? Majoritairement non, et c'est assumé : des modules CommonJS à responsabilité unique (routes → contrôleurs → services), plus simples à tester. Là où l'objet s'impose, il est utilisé : `PrismaSessionStore extends Store` (héritage réel du contrat express-session). *Preuve* : `src/config/sessionStore.js`.

Pourquoi SQLite en développement ? Zéro installation pour développer et tester. Le code Prisma est portable vers PostgreSQL, mais le provider, l'URL et surtout l'historique de migrations doivent être préparés et validés dans un environnement PostgreSQL dédié. Les pièges de portabilité du code sont traités : recherche par colonnes `*Lower` maintenues à l'écriture, agrégations hebdomadaires en JavaScript. *Preuve* : `prisma/schema.prisma` (commentaire datasource), `docs/jury/base-de-donnees.md`.

Pourquoi Prisma plutôt que du SQL brut ou un autre ORM ? Schéma déclaratif versionné, migrations générées, client typé, requêtes paramétrées par défaut (anti-injection). Le coût (dépendance lourde) est accepté pour la sécurité et la traçabilité des 13 migrations. *Preuve* : `prisma/migrations/`.

Base de données

Comment garanteez-vous l'intégrité des données ? Par la base elle-même, pas seulement par le code : unicités (`email`, `siret`, `applicationId` du contrat), clés étrangères avec cascades, contrainte composée sur les alertes. Une course à l'inscription retombe sur l'erreur `P2002`, transformée en message utilisateur. *Preuve* : `docs/jury/diagrammes/bdd-v2.png`, `test/correctifs.cjs`.

Pourquoi 8 modèles alors que le MCD initial en avait 4 ? Le cœur métier (École, Annonce, Candidature, Contrat) n'a pas bougé ; les lots ont ajouté des entités autonomes : sessions persistées, admins, alertes, journal de purge. L'évolution est documentée lot par lot. *Preuve* : `docs/jury/base-de-donnees.md` (tableau 4 → 8).

Comment passeriez-vous en production côté base ? Créer une chaîne de migrations PostgreSQL dédiée sur une base vide, la valider en préproduction, puis utiliser `npx prisma migrate deploy` et la suite de tests. Avant le

déploiement, sauvegarder avec `pg_dump` et tester la restauration avec `pg_restore` sur une base séparée. *Preuve* : `docs/jury/base-de-donnees.md`.

Sécurité

Comment protégez-vous les pièces des candidats ? Stockage privé sous `storage/` (jamais dans `public/`), noms régénérés, types vérifiés par magic bytes et non par extension, accès uniquement via des routes protégées scopées par l'école propriétaire — une école B reçoit 404 sur les documents de A. *Preuve* : `test/smoke.cjs` (cloisonnement), `src/middlewares/upload.js`.

Que se passe-t-il si quelqu'un vole un lien de suivi ? Le jeton est opaque (256 bits, non devinable) et la page ne montre aucune donnée personnelle : statut du dossier et annonce seulement. Ouvrir un lien de suivi neutralise même la session école/admin du navigateur pour empêcher une usurpation d'affichage. *Preuve* : `test/lot-b.cjs` (« aucune donnée personnelle »), `test/lot-g.cjs` (session neutralisée).

Votre signature électronique a-t-elle une valeur légale ? C'est une signature « simple » au sens eIDAS : preuve d'intégrité (empreintes SHA-256 du PDF proposé et du PDF final), horodatages et consentement explicite — suffisant pour un usage contractuel courant, mais ce n'est pas une signature qualifiée par un prestataire de confiance. L'application l'affiche : les contrats sont des modèles indicatifs à valider juridiquement. *Preuve* : `test/lot-g.cjs` (empreintes exactes), mention sous le formulaire.

Comment gérez-vous le RGPD ? Minimisation (page de suivi sans PII), consentement (double opt-in des alertes), durées de conservation appliquées par une purge automatique journalisée (7 jours / 180 jours / jetons expirés), suppression réelle au désabonnement. Le journal `PurgeRun` est montrable en démo. *Preuve* : `test/lot-j.cjs`, tuile purge sur `/admin`.

Et si l'OWASP sort une nouvelle menace ? La veille est datée et outillée : l'édition 2025 du Top 10 a été intégrée le 2026-07-10 (montée de la chaîne d'approvisionnement — réponse : dépendances minimales, lockfile, `npm audit`). La migration bcrypt → Argon2id est notée pour la production. *Preuve* : `docs/jury/veille-securite.md`.

Architecture et code

Expliquez le trajet d'une requête. Navigateur → routes (`src/routes/`) → middleware (session, CSRF, auth) → contrôleur (HTTP pur) → service (logique + Prisma) → vue Twig. Chaque couche a une responsabilité unique ; toute requête de gestion est scopée par `schoolId`. *Preuve* : arborescence `src/`, n'importe quel parcours en séance.

Comment gérez-vous les erreurs ? Erreurs utilisateur → messages de formulaire et pages 403/404/CSRF dédiées ; erreurs techniques → propagation au gestionnaire Express et page 500 sans fuite d'détails ; services externes (géocodage, Sirene, Adresse, emails) **jamais bloquants** — une panne d'API ne casse ni inscription ni publication. *Preuve* : `test/lot-f.cjs` (« API en panne → compte créé »), `test/lot-i.cjs` (« une panne d'envoi ne bloque jamais »).

Pourquoi les compteurs de vues sont-ils « fire-and-forget » ? L'affichage d'une annonce ne doit jamais attendre ni échouer à cause d'une statistique. L'incrément part sans être attendu et absorbe ses erreurs ; même philosophie pour les emails d'alerte à la publication. *Preuve* : `test/lot-h.cjs` (« increment sur id inexistant absorbe »).

Méthode

Comment avez-vous travaillé ? Chaque lot suit spécification → plan découpé en tâches → TDD (test écrit d'abord, vu échouer, implémentation minimale). Les 30 specs/plans sont versionnés, et le point de reprise `docs/jury/README.md` permet à n'importe qui de reprendre le travail — la préparation du jury elle-même a suivi ce cycle. *Preuve* : `docs/superpowers/{specs,plans}/` , historique Git.

448 assertions sans framework de test — pourquoi ? Un runner Node maison suffit : serveur dédié par fichier sur un port unique, assertions nommées, données suffixées par horodatage, nettoyage en `finally`. Zéro dépendance de test, et la suite EST la documentation des comportements. *Preuve* : `test/` (15 fichiers), `AGENTS.md` (conventions).

Qu'est-ce qui vous a posé le plus de difficulté ? La signature électronique : enchaîner validation d'image, PDF, empreintes, contre-signature et invalidation à la ré-édition sans jamais laisser un état incohérent (fichiers orphelins, contrat signé modifiable). Le test de 49 assertions a été écrit avant le code. *Preuve* : `test/lot-g.cjs` , `docs/jury/audit-certification-dwww.md` §3.

Front, accessibilité, conformité

Votre site est-il accessible ? Audit axe-core du 2026-07-10 : 0 violation sur les 15 pages après correction du contraste et des noms accessibles des graphiques ; lien d'évitement, focus visible, `aria-live`. Limite assumée : le pad de signature se dessine à la souris/au doigt — l'alternative accessible est l'import d'un fichier. *Preuve* : `docs/jury/conformite.md`.

Et la validité W3C / le responsive ? Passage final du 2026-07-11 : validateur Nu à 0 erreur et 0 avertissement sur les 15 pages, et 0 débordement sur 60 combinaisons (4 largeurs). Un faux positif responsive avait été découvert — Chromium élargissait `innerWidth` à environ 485 px pour une demande à 320 px — le contrôle utilise depuis le viewport visuel exact et l'interface possède un burger accessible. *Preuve* : `docs/jury/conformite.md` , `scripts/conformite-jury.js`.

Pourquoi pas d'application mobile ? Hors périmètre du besoin : le site web cible explicitement 320/375 px et le parcours moniteur tient en une page sans compte. Une app native n'apporterait que du coût de maintenance à ce stade. *Preuve* : `docs/jury/expression-du-besoin-v2.md` (hors-périmètre).

Production et suite

Que manque-t-il pour mettre en production ? Hébergement avec HTTPS, PostgreSQL, SMTP réel, stockage des fichiers sauvegardé, procédure de sauvegarde/restauration documentée, journalisation structurée. Le code est prêt (cookies `secure` , `trust proxy` , bascule provider) ; c'est un chantier d'infrastructure identifié, pas une réécriture. *Preuve* : `src/app.js` (branches production), audit (feuille de route).

Comment l'application encaisserait-elle la charge ? Pour un job board régional, Node/Express avec pagination systématique et requêtes indexées suffit largement. Les points chauds identifiés : géocodage et vérification SIRET — déjà mis en cache avec TTL et jamais bloquants ; au-delà, PostgreSQL + un reverse proxy avec cache statique. *Preuve* : `src/services/geocoder.js` , `siret.js` , `adresse.js` (caches).

Que feriez-vous différemment ? Prendre PostgreSQL dès le développement pour éliminer les différences de moteur (le contournement `*Lower` n'aurait pas existé), et écrire la matrice de compétences dès le début du projet plutôt qu'en préparation de maintenance. *Preuve* : limitation accents documentée dans `AGENTS.md`.

Pourquoi les moniteurs n'ont-ils pas de compte ? Choix produit central : la friction tue les candidatures. Le trio jeton de suivi + email + contre-signature en ligne couvre tout le parcours moniteur sans mot de passe à retenir ; le code refuse d'ailleurs les candidatures déposées depuis une session école ou admin. *Preuve* :
`rejectBackOfficeApplication ( src/routes/listingRoutes.js ), test/lot-c.cjs` .